Comparative Evaluation of Traditional Machine Learning and Deep Learning Models for Phishing URL Detection

Mutaganzwa Gift Desire

African Leadership University | Intro to Machine Learning Summative Project | October 2026

GitHub repository / codebase: https://github.com/mutaganzwa-desire/phishing-url-detection-ml-vs-deep-learning.git

Google Drive artifacts: Phishing_URL_ML_DL_Project

Video demonstration: https://youtu.be/CHd1K5BDUcc

ABSTRACT

Phishing URLs can lead users to credential theft, financial loss, and account compromise. This study compares six traditional machine-learning configurations with three TensorFlow/Keras multilayer perceptrons using the PhiUSIIL Phishing URL (Website) dataset from the UCI Machine Learning Repository. The downloaded dataset contained 235,795 rows. After validation and removal of 425 repeated URL values, 235,370 examples remained. To match a URL-only detection setting, 42 lexical and structural features were recomputed from each URL rather than using supplied webpage-derived features. A hostname-disjoint split was used, with 150,426 training rows, 37,609 validation rows, and 47,335 held-out test rows. Model and threshold selection used validation data. On the final test set, HistGradientBoosting achieved 99.81% accuracy and 0.9986 ROC-AUC; a regularized multilayer perceptron achieved 99.80% accuracy and 0.9988 ROC-AUC. Their performance was close, but training cost differed. A critical audit also found strong URL-format differences between classes and an unexpected high phishing score for a familiar example domain. Therefore, the results demonstrate strong within-dataset separation but do not establish real-world generalization. The study recommends controlled feature ablations and evaluation on an independent, later-collected dataset before operational use.

I. INTRODUCTION

Phishing is a cybersecurity problem because deceptive web addresses can lead users to reveal credentials, payment details, or other private information. A detection system must identify known threats and also assess URLs that have not appeared on a blocklist. URL-based classification can provide an early warning because many signals can be computed from the address without opening the destination. However, URL formats change, and models may learn accidental patterns in their training data rather than durable signals. These issues make data quality and evaluation design as important as algorithm choice [1], [3].

Existing detection approaches include blocklists, reputation and similarity checks, and machine-learning models. These approaches can complement one another, but none is sufficient in every case [1]. This study focuses on supervised binary classification from URL-string features. Its cybersecurity purpose is to explore whether a reproducible model can help flag suspicious links while keeping computation manageable. The trained models are research prototypes, not tools that guarantee a URL is safe.

The research problem is to distinguish phishing from legitimate URLs through a reproducible pipeline that does not depend on webpage-derived features that may be unavailable when only a URL is supplied. The main question is: which approach, traditional machine learning or deep learning, gives the best balance of phishing-detection performance and computational efficiency under a hostname-disjoint evaluation? A related concern is whether unusually high scores remain credible after examining the URL patterns present in the dataset.

The project trains and compares Scikit-Learn classifiers with TensorFlow/Keras neural networks on common engineered features. It audits the source data and labels, extracts numerical properties from URLs, keeps hostnames separate across training, validation, and test partitions, runs nine configurations, and selects models and thresholds using validation data. Evaluation includes precision, recall, F1-score, ROC-AUC, average

precision, confusion matrices, timing, and learning curves. The analysis weighs missed phishing URLs against false alerts.

The contribution is a controlled, logged comparison rather than a claim of a new algorithm. It uses an open dataset, independent URL feature extraction, host-disjoint evaluation, and an explicit audit of collection artifacts. The results show near-ceiling scores for both selected models, but the audit reveals very different URL formats between classes and an unusual score for a familiar example domain. Therefore, the study treats the scores as within-dataset evidence, not proof of dependable real-world detection.

II. LITERATURE REVIEW

Phishing detection includes blocklists, reputation checks, visual or textual similarity, handcrafted features, and machine-learning classification. Zieni, Massari, and Calzarossa survey these approaches and show that results depend on both the detection method and dataset [1]. Blocklists work well for known indicators but struggle with newly registered or modified domains. Machine learning can generalize from examples, but it depends on representative data and stable relationships between input features and labels. Scores must therefore be interpreted in the context of how samples were collected.

Traditional methods have shown strong results with URL and website features. Alani and Tawfik's PhishNot uses a reduced feature set and reports high Random Forest performance while also considering speed and cloud deployment [4]. Ahammad et al. compare machine-learning methods for phishing URL detection, including gradient-boosting approaches [5]. Work on lexical URL features has also examined tuned LightGBM models and feature selection, emphasizing that URL-only inputs can be faster and less dependent on external services than methods that fetch webpage content [6]. These studies justify traditional models as important baselines, but their results do not prove transfer to future URLs or different collection sources.

Sánchez-Paniagua et al. address a data realism problem by collecting login-page URLs from both legitimate and phishing sites and by examining performance across collection years [7]. Their work matters because benign homepages alone may not represent legitimate login URLs. If legitimate and malicious examples differ systematically in path, format, or source age, a model may learn a feature that identifies the collection process rather than the threat itself. This concern is directly relevant to URL classification.

Deep-learning methods can learn patterns from URL characters or words instead of relying only on manually designed statistics. URLNet combines character-level and word-level convolutional representations for malicious URL detection [8]. This can capture sequence patterns that fixed features may miss. However, deep models need careful regularization and often cost more to train. The present project uses multilayer perceptrons (MLPs) over engineered numerical features, not the character-level CNN proposed by URLNet. Its findings therefore describe this tabular MLP comparison and should not be generalized to every deep-learning architecture.

The PhiUSIIL dataset, developed by Prasad and Chandra, contains 235,795 URL and webpage examples and supports similarity-based and incremental detection [2]. UCI reports 134,850 legitimate and 100,945 phishing records, with original labels 1 for legitimate and 0 for phishing [9]. The dataset is suitable for this assignment because it is open, labeled, and much larger than 20,000 rows. Its mixed feature sources also require care: URL-title similarity and other webpage-derived fields are not equivalent to features that can be computed from a URL alone. Using them in a URL-only detector could create an input-availability mismatch or shortcut.

Rashid et al. study cross-dataset generalization and report that phishing URL detectors can lose substantial performance when tested on a source different from the one used for training; some reported drops are around 10% to 32% [3]. They relate the drop to distribution changes, including differences introduced by how URLs are collected and formatted. This motivates a hostname-disjoint split and a URL-format audit in the current work. Host separation reduces overlap, but it is not a true external-source or temporal test and cannot remove every collection artifact.

ROC analysis measures ranking over true-positive and false-positive rates [10]. Precision and recall add operational meaning: recall measures the share of phishing URLs caught, while precision measures how many flagged URLs are truly phishing. Davis and Goadrich explain the connection between ROC and precision-recall curves [11], and Saito and Rehmsmeier show that precision-recall plots can be more informative under class imbalance [12]. Although the PhiUSIIL classes are not severely imbalanced, average precision remains useful for comparing performance across thresholds.

The model families differ in assumptions and costs. Random forests combine randomized decision trees and model nonlinear feature interactions [13], while histogram-based gradient boosting builds trees sequentially and is intended to work efficiently on larger tabular datasets [14]. Neural networks learn layered nonlinear transformations, often using regularization and early stopping to limit overfitting [15]. Thus, the relevant question is empirical: whether additional model complexity improves the result enough to justify its cost.

The literature supports three design choices: use URL-only features to match a lightweight detector's available input, keep hostnames separate between splits, and evaluate both ranking and threshold-specific errors. The project's gap is not the absence of another classifier; it is the need for a reproducible same-feature comparison with an explicit check for dataset shortcuts. This study addresses that gap within one benchmark. Independent and time-based testing remain necessary before making stronger generalization claims.

III. METHODOLOGY

The study uses a quantitative experimental design and the official PhiUSIIL dataset from the UCI Machine Learning Repository, dataset ID 967. The executed notebook reports a successful source ZIP download and 235,795 raw rows across 56 columns. The target contained 100,945 records with original label 0 and 134,850 with original label 1. Following UCI, original label 0 was mapped to phishing and original label 1 to legitimate; the project defined phishing as the positive class [9]. Correct label mapping is essential for interpreting recall and confusion matrices.

The raw audit found no missing cells and no exact duplicate rows. There were 425 repeated URL values, and no URL with conflicting labels was detected. After validating URLs and removing repeated values, 235,370 rows remained: 134,850 legitimate and 100,520 phishing examples. This remains far above the assignment threshold. The source includes URL-derived and webpage-derived fields, but the predictive models did not use copied webpage features. Instead, their input features were independently recomputed from the URL string.

The extractor created 42 numerical features covering URL, host, path, query and fragment length; counts of dots, hyphens, underscores, at-signs, question marks, equals signs, ampersands, percent symbols and slashes; letter, digit and special-character counts and ratios; approximate subdomain count and top-level-domain length; query-parameter count; HTTPS and HTTP indicators; `www`, IP-address, port, punycode, percent-encoding and fragment indicators; and character entropy for the URL and host. It also recorded host digit and hyphen ratios and the presence and count of terms such as login, verify, account, password, billing, and support. These features require no request to the destination website.

The feature matrix was checked for missing and infinite values; neither was found. Logistic Regression used median imputation and StandardScaler, with an optional mutual-information SelectKBest step. Tree methods used median imputation without scaling. For neural networks, the imputer and scaler were fitted only on training data and then applied unchanged to validation and test data. The cleaning and feature-extraction steps were deterministic under seed 42. The notebook also saved the feature schema and target mapping in its model metadata.

A two-stage stratified group split used a normalized hostname as the group. The final partitions contained 150,426 training rows (63.91%), 37,609 validation rows (15.98%), and 47,335 test rows (20.11%). The notebook verified zero hostname overlap between partitions. The training split fitted the models and preprocessing; validation compared configurations, selected one model per family, and selected the decision

thresholds. The held-out test set was used for the final reported comparison. This is more careful than a random row split, but it does not replace testing on a separate source or future time period.

Six traditional configurations were evaluated. ML01 was scaled to Logistic Regression with C=0.1 and a maximum of 1,500 iterations. ML02 used C=1.0 and mutual-information selection of the top 15 features. ML03 used a Random Forest with 140 trees, unlimited depth, leaf size 1, and square-root feature sampling. ML04 used 140 trees with depth limited to 20, leaf size 3, and 0.8 feature sampling. ML05 used Extra Trees with 160 trees, leaf size 2, and square-root feature sampling. ML06 used histogram-based gradient boosting with learning rate 0.08, up to 160 iterations, 31 leaf nodes, L2 regularization 1.0, and early stopping. These settings tested linear separation, feature reduction, tree flexibility, randomized splits, and boosting [13], [14].

Three TensorFlow/Keras MLPs used the Sequential API, ReLU hidden layers, and a sigmoid output. DL07 used 64 and 32 units without dropout or L2. DL08 used 128, 64, and 32 units, dropout 0.25, L2=0.0001, and learning rate 0.001. DL09 used 128 and 64 units, dropout 0.15, L2=0.0001, and learning rate 0.0003. All used Adam, binary cross-entropy, batch size 512, and `tf.data` datasets with batching and prefetching. Early stopping monitored validation loss with patience four and restored the best weights. ReduceLROnPlateau halved the learning rate after a validation-loss plateau, with patience two and a minimum of 0.00001 [15].

The notebook fixed random seeds at 42 and recorded Python 3.13.16, pandas 2.2.3, TensorFlow 2.21.0, and a Linux Colab runtime with no detected GPU. All nine experiments were recorded in CSV and JSON, with scalar results and MLP histories sent to TensorBoard. The saved artifacts included the selected models, neural preprocessing objects, feature schema, metadata, experiment tables, and raw dataset ZIP. The completed artifact folder was exported to the Google Drive link shown on the first page. Timings are specific to this CPU runtime and should be interpreted as within-run comparisons.

Metrics included accuracy, balanced accuracy, precision, recall, F1, specificity, ROC-AUC, average precision, and confusion-matrix counts. A false negative is a phishing URL classified as legitimate; a false positive is a legitimate URL flagged as phishing. Each family's model was selected using validation PR-AUC, with ROC-AUC as a tie-break. The threshold was selected to maximize validation F1, then applied to the final holdout. This avoids choosing models from test scores, although results from one split still have uncertainty and can reflect shared dataset bias.

IV. RESULTS

The downloaded dataset met the size and classification requirements. Figure 1 shows 134,850 legitimate and 100,945 phishing records before deduplication, equal to 57.19% and 42.81%. After removing repeated URL values, 100,520 phishing examples remained. All experiments used the same 150,426-row training set and 37,609-row validation set. Table I summarizes the dataset, cleaning, features, and split. Shared partitions make the model configurations more directly comparable.

TABLE I. DATASET AND SPLIT SUMMARY

Item	Observed value	Interpretation
Raw UCI rows	235,795	Above 20,000-row requirement
Raw source columns	56	Includes URL, filename, predictors, target
Raw legitimate / phishing	134,850 / 100,945	57.19% / 42.81%
Repeated URLs removed	425	No conflicting URL labels reported
Rows after cleaning	235,370	All records used URL-based feature extraction
Engineered features	42	Computed from URL string only
Train / validation / test	150,426 / 37,609 / 47,335	Hostname overlap across splits: zero

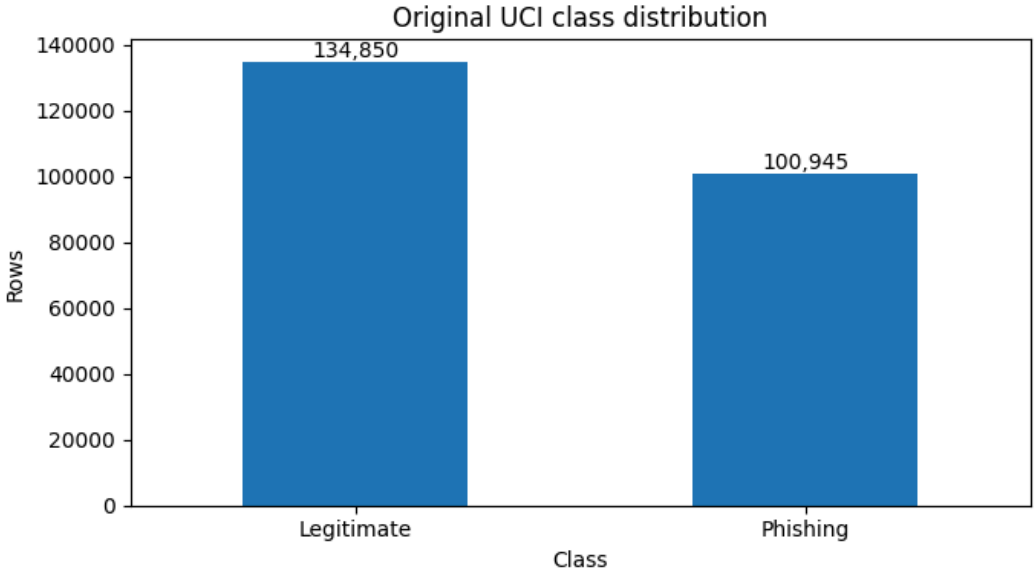


Fig. 1. Original class distribution in the downloaded UCI dataset before URL deduplication.

The URL audit showed strong class-specific formatting. All legitimate strings in the raw table started with `https://` and `https://www.`, while only 48.74% of phishing URLs used HTTPS and 2.43% started with HTTPS plus `www`. A non-root path appeared in 27.20% of phishing URLs but none of the legitimate URLs; query strings appeared in 6.02% of phishing URLs and none of the legitimate URLs. Median URL length was 34 characters for phishing and 27 for legitimate URLs. The precomputed URLSimilarityIndex equaled 100 for every legitimate record but only 0.78% of phishing records. These precomputed fields were excluded from the model features, but the raw format differences remain a validity concern.

TABLE II. SELECTED URL-FORMAT DIFFERENCES OBSERVED IN THE RAW DATA

Audited characteristic	Phishing	Legitimate
Starts with https://	48.74%	100.00%
Starts with https://www.	2.43%	100.00%
Contains non-root path	27.20%	0.00%
Contains query string	6.02%	0.00%
Median URL length	34 characters	27 characters
URLSimilarityIndex = 100	0.78%	100.00%

Table III summarizes the nine validation experiments. The selected HistGradientBoosting model reached ROC-AUC 0.9986 and PR-AUC 0.9989; the selected regularized MLP reached 0.9986 and 0.9989. Logistic Regression obtained ROC-AUC 0.9985 and PR-AUC 0.9989 while fitting in 2.45 seconds, compared with 9.79 seconds for HistGradientBoosting and 92.05 seconds for the selected MLP. The flexible Random Forest was weaker, with PR-AUC 0.9981 and ROC-AUC 0.9982. Feature-selection Logistic Regression did not improve on the baseline's ranking metrics.

TABLE III. NINE VALIDATION EXPERIMENTS

ID / model	Main experimental setting	Val. ROC-AUC	Val. PR-AUC	Fit (s)	ms / 1,000
ML01 Logistic baseline	C=0.1; scaled; 1,500 max iterations	0.9985	0.9989	2.45	2.01
ML02 Logistic + selection	C=1.0; mutual-info top 15	0.9984	0.9988	49.26	1.83

ID / model	Main experimental setting	Val. ROC-AUC	Val. PR-AUC	Fit (s)	ms / 1,000
ML03 Random Forest flexible	140 trees; unlimited depth; leaf=1	0.9982	0.9981	20.32	11.92
ML04 Random Forest regularized	140 trees; depth=20; leaf=3	0.9985	0.9989	64.65	16.12
ML05 Extra Trees	160 trees; leaf=2; sqrt features	0.9985	0.9989	17.65	17.27
ML06 HistGradientBoosting	lr=0.08; 160 iter; 31 leaves; L2=1	0.9986	0.9989	9.79	22.15
DL07 MLP baseline	[64,32]; dropout=0; L2=0; lr=0.001	0.9986	0.9989	27.18	7.65
DL08 MLP regularized	[128,64,32]; dropout=0.25; L2=1e-4	0.9986	0.9989	92.05	7.65
DL09 MLP lower LR	[128,64]; dropout=0.15; L2=1e-4; lr=0.0003	0.9986	0.9989	95.45	10.24

All nine runs used the same train and validation partitions: 150,426 and 37,609 rows respectively. The MLPs used a batch size of 512, Adam, binary cross-entropy, `tf.data`, early stopping, and learning-rate reduction on a validation-loss plateau. In the table, “ms / 1,000” is the measured prediction time per 1,000 validation rows from this CPU-based runtime. These timings depend on the environment and should not be read as hardware-independent benchmarks.

Table IV presents final test performance. HistGradientBoosting achieved 99.81% accuracy, 99.97% precision, 99.59% recall, 99.78% F1, ROC-AUC 0.9986, and average precision 0.9990. The MLP achieved 99.80% accuracy, 99.93% precision, 99.61% recall, 99.77% F1, ROC-AUC 0.9988, and average precision 0.9991. The differences are small: the neural model ranks examples slightly better and has marginally higher recall, while the traditional model has slightly higher accuracy, precision, and F1. These metrics do not establish a clear overall winner.

The validation-selected threshold was 0.6001 for HistGradientBoosting and 0.1912 for the MLP. The different values reflect different score distributions. They were chosen to maximize validation F1, not to meet a security organization's specific cost target. A system that places greater cost on missed phishing URLs may require a different threshold, but its higher false-alert rate should be assessed as well.

TABLE IV. FINAL HOLDOUT METRICS FOR THE VALIDATION-SELECTED MODELS

Metric	HistGradientBoosting	Regularized MLP
Validation-selected threshold	0.6001	0.1912
Accuracy	99.81%	99.80%
Balanced accuracy	99.79%	99.78%
Precision	99.97%	99.93%
Recall (phishing detection rate)	99.59%	99.61%
F1-score	99.78%	99.77%
Specificity	99.98%	99.95%
ROC-AUC	0.9986	0.9988
Average precision (PR-AUC)	0.9990	0.9991
Training time	9.79 s	92.05 s
Prediction time / 1,000 URLs	22.15 ms	7.65 ms

Figure 2 and Table V show that HistGradientBoosting produced 26,868 true negatives, 6 false positives, 83 false negatives, and 20,378 true positives. The MLP produced 26,860 true negatives, 14 false positives, 79 false negatives, and 20,382 true positives. The test set contained 20,461 phishing and 26,874 legitimate URLs. The MLP detected four additional phishing examples but flagged eight more legitimate URLs. The small recall difference should not be exaggerated. False negatives can expose users to malicious destinations, while false positives may disrupt normal work and reduce trust in warnings.

TABLE V. CONFUSION-MATRIX COUNTS (PHISHING IS THE POSITIVE CLASS)

Model	TN	FP	FN	TP
HistGradientBoosting	26,868	6	83	20,378
Regularized MLP	26,860	14	79	20,382

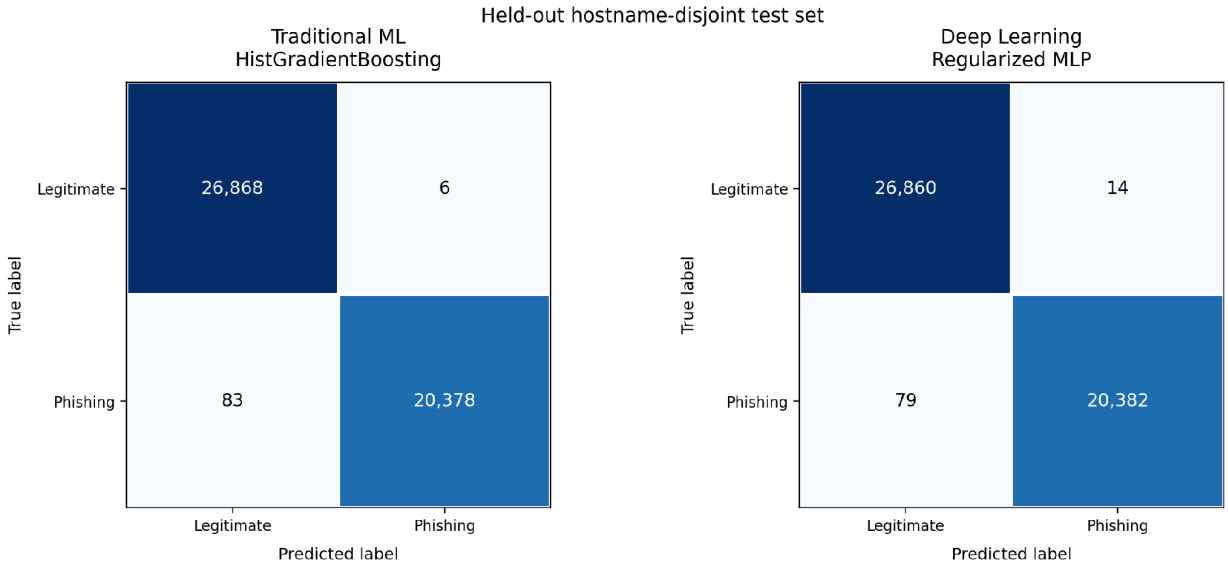


Fig. 2. Confusion matrices on the held-out hostname-disjoint test set at validation-selected thresholds. Exact cell counts are shown.

Figure 3 shows both test ROC curves near the upper-left corner and precision-recall curves close to precision 1 for most of the displayed recall range. This matches the near-ceiling AUC values, but a plot at this scale can hide the operational significance of a small number of mistakes. Further, the results come from one split of one source dataset. They do not show that performance will remain high on later data or URLs collected by another organization.

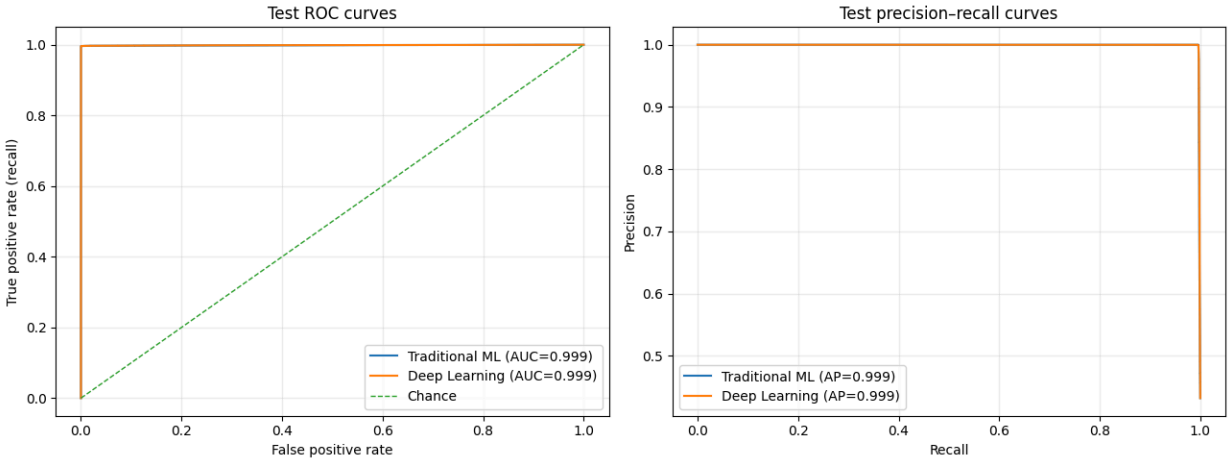


Fig. 3. Test ROC curves (left) and precision-recall curves (right) for the selected traditional and deep-learning models.

In Figure 4, MLP training and validation loss fall quickly and converge near 0.015. Training and validation ROC-AUC remain close, around 0.9982–0.9984, without a sustained divergence. Figure 5 shows Logistic Regression training ROC-AUC rising from about 0.99829 at roughly 15,000 training rows to a peak near 0.99847; validation stays around 0.99850–0.99861. Additional training rows make little visible difference. The learning curves show no obvious classic overfitting pattern, but shared data bias can raise both training and validation results.

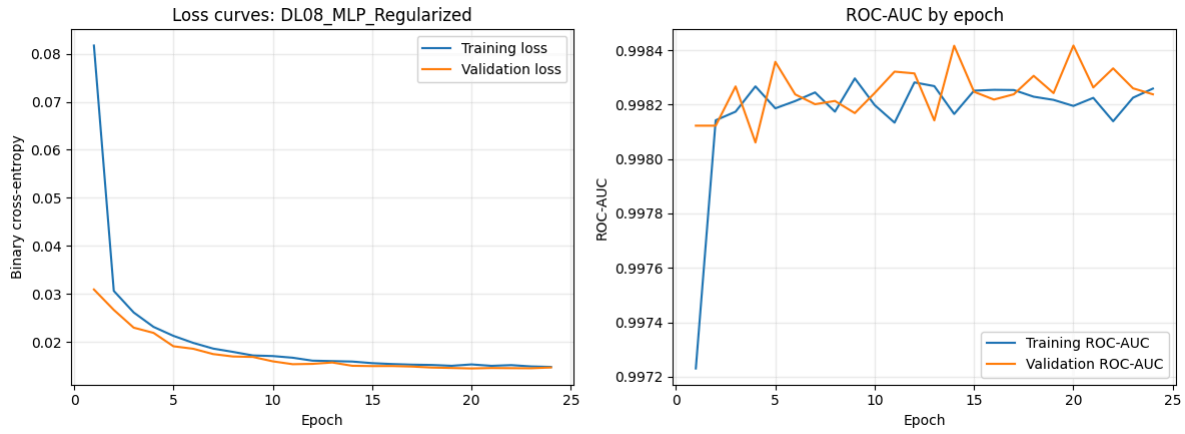


Fig. 4. Training and validation loss and ROC-AUC by epoch for the selected regularized MLP.

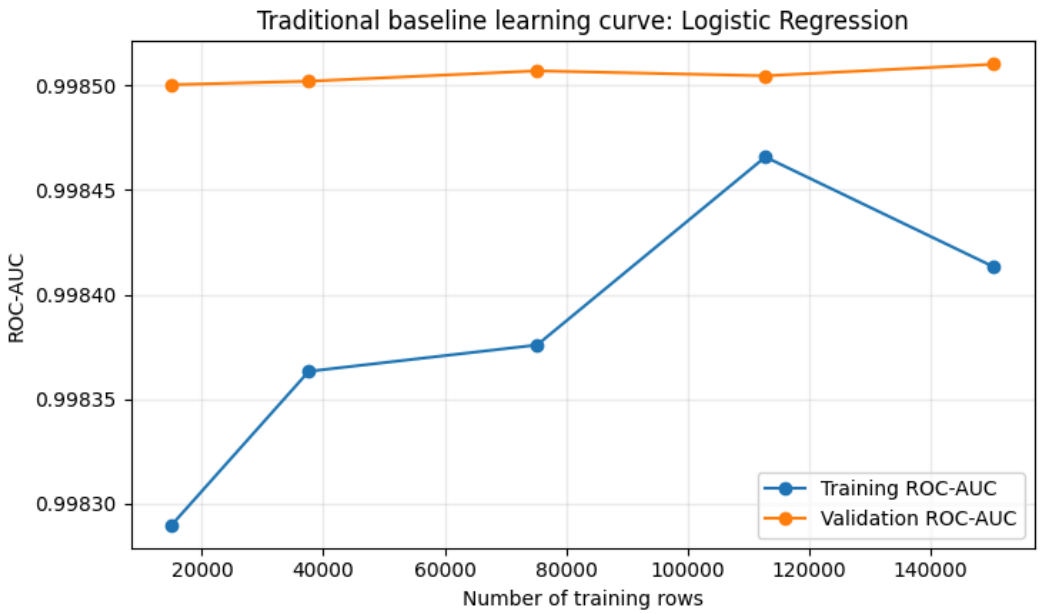


Fig. 5. Logistic Regression learning curve across increasing training-set sizes.

The feature-importance cell did not produce a ranking because the selected HistGradientBoosting estimator did not provide the direct attribute expected by the function. No feature-importance plot is claimed. The notebook saved selected-model test errors, although its visible first 20-row preview begins with the traditional model and is not a balanced sample from both models. The inference example `https://www.example.com/` received phishing probabilities 0.999312 from HistGradientBoosting and 0.997903 from the MLP. The notebook did not establish a verified label for that exact input, so this is not counted as a formal false positive. It is a strong sanity-check warning because the URL resembles the format associated with legitimate samples.

V. DISCUSSION

The primary result is that the traditional model and the tested MLP performed almost identically on the host-disjoint test set. This answers the question only for this dataset and feature contract. It does not show that the paradigms are equivalent on all phishing tasks. A large feature set can make class separation straightforward, allowing linear and tree-based models to learn most of the available signal. A deeper network's additional capacity may add little when the predictive relationship is already strong or driven by simple data regularities.

Computational cost favors simpler methods in this run. The MLP took 92.05 seconds to train, more than nine times the 9.79 seconds for HistGradientBoosting and around 38 times the 2.45 seconds for Logistic Regression. Its prediction time per 1,000 URLs was 7.65 ms, lower than the 22.15 ms measured for HistGradientBoosting but higher than the 2.01 ms baseline. These values depend on the runtime, and the experiment used no GPU. Nevertheless, when performance is nearly equal, the simpler model has a clear training-cost advantage. A final recommendation must wait until a more realistic dataset test confirms that its error profile remains acceptable.

The most serious concern is shortcut learning. In the raw dataset, every legitimate URL began with HTTPS and `www`, and none had a path or query string in the audit, while phishing records differed substantially. These are not universal truths about the web: legitimate websites often have paths and query parameters, and phishing sites can use HTTPS. The classifier may therefore rely on a proxy for the label that works in this sample but fails elsewhere. Very high scores are compatible with this problem and should not be interpreted as proof that the model learned general phishing characteristics.

The `example.com` inference check reinforces the concern. The notebook assigns very high phishing probabilities to an address in a familiar legitimate-looking format. Since no ground-truth check was performed, it is incorrect to report this as a confirmed error. It should instead trigger controlled checks of URLs with and without a trailing slash, other scheme and `www` variants, and known benign addresses. The exact 42-feature vectors should be inspected. Permutation importance on validation data should then measure the effect of shuffling each feature. Such rankings describe model dependence, not causation, and correlated features can affect their interpretation.

Hostname-disjoint splitting prevents one hostname from appearing in more than one split, but all partitions still came from the same dataset pipeline. It does not reproduce time-based change, variation between security feeds, or URL transformations by scanners and email gateways. Rashid et al. report substantial drops in cross-dataset phishing performance [3], and Sánchez-Paniagua et al. show why realistic login URLs and temporal testing matter [7]. The next study should test on a separately sourced, later-collected dataset with labels verified under a clear policy. The current holdout should not be used repeatedly while modifying the model, because it would gradually become a validation set.

The learning curves show that obvious overfitting is not the main visible problem: train and validation curves remain close. But a small gap does not establish low bias or reliable generalization when both subsets share the same collection artifacts. Learning curves diagnose optimization and data-size behavior within the current distribution; they cannot detect all forms of dataset bias. This distinction is essential when interpreting the high test metrics.

F1 was used for threshold selection, but operational thresholds should reflect the cost of errors. A security team may value recall more than precision because missed phishing links can result in stolen credentials. However, false positives can block legitimate banking, educational, or public-service websites. Further tests should report alerts per 1,000 benign URLs, missed phishing URLs per 1,000 attacks, precision-recall trade-offs, calibration, and preprocessing-inclusive latency. These measures would support a practical decision better than a single threshold or accuracy value.

Limitations include reliance on one benchmark, systematic class-specific formatting, no independent or time-based test, no calibration evaluation, a missing feature-importance plot, and an unresolved inference sanity check. The nine experiments meet the minimum experimental count but do not constitute exhaustive optimization. Runtime measurements also come from one CPU-based Colab execution. These limits narrow the conclusion; they do not erase the value of the recorded model comparison. The next experiments should compare a simple formatting-rule baseline with the full model, normalize or remove selected format-sensitive features in controlled ablations, generate permutation importance, and run balanced error review. Major changes should be confirmed on a fresh independent test set.

VI. CONCLUSION

This project compared six traditional ML configurations and three TensorFlow/Keras MLPs for phishing URL classification. The UCI PhiUSIIL source met the dataset requirements with 235,795 raw rows. After validation and removal of 425 repeated URL values, 235,370 examples and 42 independently engineered URL features remained. A hostname-disjoint split gave 150,426 training rows, 37,609 validation rows, and 47,335 test rows. Models and thresholds were selected using validation data, leaving the test set for the final comparison.

On the holdout, HistGradientBoosting achieved 99.81% accuracy, 99.97% precision, 99.59% recall, and ROC-AUC 0.9986. The regularized MLP achieved 99.80% accuracy, 99.93% precision, 99.61% recall, and ROC-AUC 0.9988. Average precision was 0.9990 and 0.9991, respectively. The MLP missed four fewer phishing URLs but produced eight additional false positives, and it took much longer to train. Logistic Regression was almost as strong on validation while fitting faster, which shows that a more complex model is not automatically necessary for these features.

The main conclusion is about validity. Strong class-specific URL formatting and an unexpected high risk score for a familiar example domain raise concern that the classifiers may learn dataset shortcuts. The experiment supports strong separation within this host-disjoint holdout, but not yet reliable detection on new campaigns or general internet traffic.

Future work should include controlled feature ablations, a simple formatting-rule baseline, permutation importance, balanced error review, calibration, and testing on an independent dataset collected later. Until those tests are completed, model simplicity and stronger external validation matter more than the small difference in AUC. The notebook offers a reproducible starting point for a careful comparison of traditional ML and deep learning for cybersecurity, while the dataset audit defines the limits of its current evidence.

REFERENCES

[1] R. Zieni, L. Massari, and M. C. Calzarossa, "Phishing or Not Phishing? A Survey on the Detection of Phishing Websites," IEEE Access, vol. 11, pp. 18499–18519, 2023, doi: 10.1109/ACCESS.2023.3247135.

[2] A. Prasad and S. Chandra, "PhiUSIIL: A diverse security profile empowered phishing URL detection framework based on similarity index and incremental learning," Computers & Security, vol. 136, Art. no. 103545, 2024, doi: 10.1016/j.cose.2023.103545.

[3] F. Rashid, B. Doyle, S. C. Han, and S. Seneviratne, "Phishing URL detection generalisation using Unsupervised Domain Adaptation," Computer Networks, vol. 245, Art. no. 110398, 2024, doi: 10.1016/j.comnet.2024.110398.

[4] M. M. Alani and H. Tawfik, "PhishNot: A cloud-based machine-learning approach to phishing URL detection," Computer Networks, vol. 218, Art. no. 109407, 2022, doi: 10.1016/j.comnet.2022.109407.

[5] S. K. H. Ahammad, S. D. Kale, G. D. Upadhye, S. D. Pande, E. V. Babu, A. V. Dhumane, and D. K. Jang Bahadur, "Phishing URL detection using machine learning methods," Advances in Engineering Software, vol. 173, Art. no. 103288, 2022, doi: 10.1016/j.advengsoft.2022.103288.

[6] "Phishing Website Detection Using LGBM Classifier With URL-Based Lexical Features," in Proc. 2022 IEEE Silchar Subsection Conference (SILCON), 2022, doi: 10.1109/SILCON55242.2022.10028793.

[7] M. Sánchez-Paniagua, E. Fidalgo, E. Alegre, W. Al-Nabki, and V. González-Castro, "Phishing URL detection: A real-case scenario through login URLs," IEEE Access, vol. 10, pp. 42949–42960, 2022, doi: 10.1109/ACCESS.2022.3168681.

[8] H. Le, Q. Pham, D. Sahoo, and S. C. H. Hoi, "URLNet: Learning a URL representation with deep learning for malicious URL detection," arXiv:1802.03162, 2018. [Online]. Available: https://arxiv.org/abs/1802.03162.

[9] A. Prasad and S. Chandra, "PhiUSIIL Phishing URL (Website) Dataset," UCI Machine Learning Repository, Dataset 967, 2024, doi: 10.1016/j.cose.2023.103545. [Online]. Available: https://archive.ics.uci.edu/dataset/967/phiusiil+phishing+url+website+dataset.

[10] T. Fawcett, "An introduction to ROC analysis," Pattern Recognition Letters, vol. 27, no. 8, pp. 861–874, 2006, doi: 10.1016/j.patrec.2005.10.010.

[11] J. Davis and M. Goadrich, "The relationship between Precision-Recall and ROC curves," in Proc. 23rd Int. Conf. Machine Learning (ICML), 2006, pp. 233–240, doi: 10.1145/1143844.1143874.

[12] T. Saito and M. Rehmsmeier, "The precision-recall plot is more informative than the ROC plot when evaluating binary classifiers on imbalanced datasets," PLOS ONE, vol. 10, no. 3, Art. no. e0118432, 2015, doi: 10.1371/journal.pone.0118432.

[13] L. Breiman, "Random forests," Machine Learning, vol. 45, pp. 5–32, 2001, doi: 10.1023/A:1010933404324.

[14] F. Pedregosa et al., "Scikit-learn: Machine learning in Python," Journal of Machine Learning Research, vol. 12, pp. 2825–2830, 2011.

[15] I. Goodfellow, Y. Bengio, and A. Courville, Deep Learning. Cambridge, MA, USA: MIT Press, 2016.

Reproducibility and scope note. *The numerical findings in this report correspond to the uploaded executed notebook. The feature-importance chart was not generated in that run, the example URL prediction has not been independently ground-truthed, and an external or time-based dataset was not evaluated. These issues are treated as limitations rather than hidden or replaced with invented results.*